

Final Exam Topics

4.1 Coding theory

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

1 Coding theory

1.1 Letter-by-letter coding

Coding, in the most general sense, means the mapping of the set of messages into another set. Often the message is given by a sequence formed from the elements of some character set. Then we decompose the message into predetermined elementary parts such that every message is uniquely produced as a sequence of such elementary parts. For the coding we give the code of the elementary parts, which a dictionary contains. Such coding is called letter-by-letter coding.

The messages to be coded are letters of an alphabet A , and the code of each letter corresponds to letters of another alphabet B (code alphabet). Suppose that both alphabets are nonempty and finite.

We denote the set of words that can be written from the letters of an alphabet A by A^+ , and the one extended with the empty word by A^* .

Based on this, letter-by-letter coding is determined by a map $\varphi : A \rightarrow B^*$, which we can extend to a map $\psi : A^* \rightarrow B^*$ as follows: If $\alpha_1\alpha_2\dots\alpha_n = \alpha \in A$, then the code of α is $\psi(\alpha) = \varphi(\alpha_1)\varphi(\alpha_2)\dots\varphi(\alpha_n)$. Clearly, if φ is not injective (or the empty word is in its range), then the coding ψ is also not injective, that is, not uniquely decodable. Because of this we may assume that φ is injective and maps into B^+ .

1.2 Shannon and Huffman codes

Basic concepts

- Frequency, relative frequency, distribution

The information source emits n messages. We denote the different messages by $a_1, \dots, a_m$. The message a_i occurs k_i times, which we call its frequency. The relative frequency of a_i is $p_i = k_i/n$. The m -tuple of numbers $p_1, \dots, p_m$ is called the distribution of the messages. ($\sum_{i=1}^m p_i = 1$)

- Information content

The individual information content of the message a_i is $I_i = -\log_r p_i$, where $r > 1$ is the unit of information. (For $r = 2$ the unit is the bit).

- Entropy

The average information content emitted by the message source is called entropy:

$$H_r(p_1, \dots, p_m) = - \sum_{i=1}^m p_i \log_r p_i$$

- Prefix, suffix, infix

Let $\alpha, \beta, \gamma \in A$ be words. Then for the word $\alpha\beta\gamma$, α is a prefix, β an infix, and γ a suffix.

- Code tree

For letter-by-letter coding an illustrative directed, edge-labeled tree can be unambiguously given. Let $\varphi : A \rightarrow B^*$ be the letter-by-letter coding. Let us construct a tree whose root is the empty word and such that if $\beta = \alpha b$ ($b \in B$), then from α there is an edge to β with label b . Then all words of the same length will be on one level. The vertices from which an edge leads out with every label $b \in B$ are called full vertices, otherwise truncated vertices.

- Prefix code, uniform code, comma code

The letter-by-letter coding $\psi : A^* \rightarrow B^*$ determined by the injective map $\varphi : A \rightarrow B^+$ is

1. decodable (uniquely decodable) if ψ is injective
2. a prefix code if the range of φ is prefix-free.
3. a uniform code (fixed length) if every element in the range of ψ has the same length
4. a comma code if $\exists \vartheta \in B^+$ comma such that ϑ is a suffix of every code word, but is neither a prefix nor an infix of any code word.

- Average word length

Let $A = \{a_1, \dots, a_n\}$ be the alphabet to be coded. The length of the code of a_i is l_i . Then $\bar{l} = \sum_{i=1}^n p_i l_i$ is the average word length of the code.

- Optimal code

If, for a given alphabet size and given distribution, the average word length of a decodable letter-by-letter code is minimal, then it is called an optimal code.

Shannon code

The Shannon code is an optimal code (with an alphabet of size r and frequencies p_i), which we construct as follows.

1. Order the letters in decreasing order of their relative frequencies.
2. Determine the word lengths $l_1, \dots, l_n$ as follows:

$$r^{-l_i} \leq p_i < r^{-l_i+1}$$

3. Distribute the elements of the alphabet over the individual place values.

Example:

Let the code alphabet be the set 0, 1, 2, and the letters to be coded and their frequencies be the following:

a	b	c	d	e	f	g	h	i	j
0.17	0.02	0.13	0.02	0.01	0.31	0.02	0.17	0.06	0.09

After ordering the relative frequencies:

f	a	h	c	j	i	b	d	g	e
0.31	0.17	0.17	0.13	0.09	0.06	0.02	0.02	0.02	0.01

Determine the word lengths. For f, a, h and c: $3^{-2} = r^{-l_i} \leq p_i < r^{-l_i+1} = 3^{-1}$ So their word length is 2.

We proceed the same way in the other cases:

f	a	h	c	j	i	b	d	g	e
0.31	0.17	0.17	0.13	0.09	0.06	0.02	0.02	0.02	0.01
2	2	2	2	3	3	4	4	4	5

Based on these, the code word of f is 00, of a is 01, h has 02, while c has 10. After these, j would have 11, but since that is of length 3, it is 110. The code words thus turn out as follows:

f	a	h	c	j	i	b	d	g	e
0.31	0.17	0.17	0.13	0.09	0.06	0.02	0.02	0.02	0.01
2	2	2	2	3	3	4	4	4	5
00	01	02	10	110	111	1120	1121	1122	12000

The code tree can be seen in Figure 1.

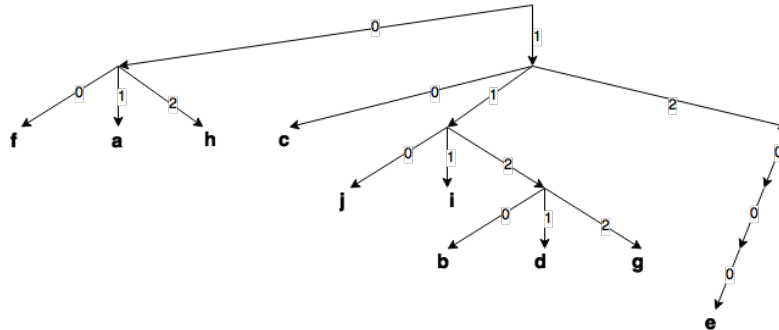


Figure 1: Example code tree of the Shannon code

Huffman code

The Huffman code is also an optimal code (with an alphabet of size r and frequencies p_i), which we construct as follows.

1. Order the letters in decreasing order of their relative frequencies.
2. In order that only one truncated vertex is created, the congruence

$$m \equiv n \pmod{r-1}$$

must hold, where m is the out-degree of the single truncated vertex. This is equivalent to $m = 2 + ((n-2) \bmod r - 1)$. So divide $n - 2$ by $r - 1$, and thus m will be the remainder $+ 2$.

3. In the first step we merge the last m letters of the sequence (give it a new notation/letter), and its relative frequency will be the sum of the relative frequencies of the terms. Order the sequence. After this step the number of letters is already congruent to $r - 1$, so in the following reduction steps we can always create full vertices.
4. Merge the last r letters, replace them with a new letter and let its relative frequency be the sum of the relative frequencies.
5. Repeat the reduction step in 4 until r letters remain. Then we assign to each letter in turn a letter of the code alphabet.
6. If we encounter a reduced element we split it, then assign the letters of the code alphabet to its elements as well, but concatenate them with the previous one.
7. Repeat the step in 6 as long as a reduced element remains.

Example:

Let us code the source seen at the Shannon code the same way with the $\{0, 1, 2\}$ code alphabet.

a	b	c	d	e	f	g	h	i	j
0.17	0.02	0.13	0.02	0.01	0.31	0.02	0.17	0.06	0.09

Order by relative frequency:

f	a	h	c	j	i	b	d	g	e
0.31	0.17	0.17	0.13	0.09	0.06	0.02	0.02	0.02	0.01

Divide $n - 2$ by $r - 1$: $10 - 2 = 4 * (3 - 1) + 0$. So m is the remainder + 2, that is $m = 2$. We merge the last m letters, and order the sequence:

f	a	h	c	j	i	(g,e)	b	d
0.31	0.17	0.17	0.13	0.09	0.06	0.03	0.02	0.02

From here on, in every reduction step we merge the last r , i.e. 3 letters:

f	a	h	c	j	((g,e), b, d)	i
0.31	0.17	0.17	0.13	0.09	0.07	0.06

We repeat this until r letters remain:

(a,h,c)	f	(j,((g,e),b,d),i)
0.47	0.31	0.22

Based on the splitting we can assemble the tree shown in Figure 2.

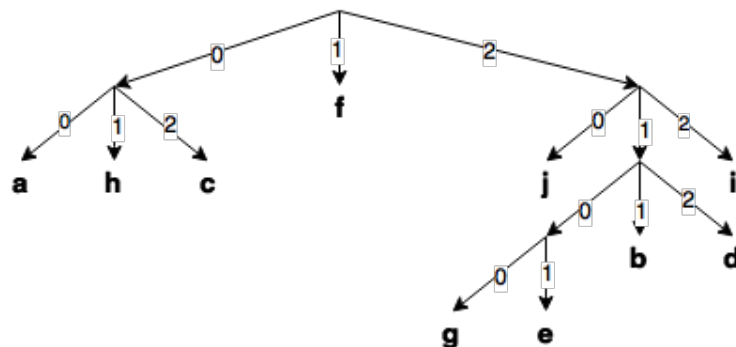


Figure 2: Example code tree of the Huffman code

Based on these, the code table:

letter	frequency	code
f	0.31	1
a	0.17	00
h	0.17	01
c	0.13	02
j	0.09	20
i	0.06	22
b	0.02	211
d	0.02	212
g	0.02	2100
e	0.01	2101

1.3 Error-correcting codes, code distance

Error-limiting coding

Error-limiting codes can be sorted into two groups: error-detecting and error-correcting codes. In both cases we assign code words to the messages, based on which we can handle the errors arising during transmission. If the message is easily repeatable we use an error-detecting code, if it is hard to repeat an error-correcting code. With error-limiting codes we always use code words of the same length.

Distance, weight of codes

The Hamming distance $d(u, v)$ of the words u and v of the code alphabet is the number of differing digits in the same position. The Hamming distance has the usual properties of a distance, that is, $\forall u, v, z$:

- $d(u, v) \geq 0$

- $d(u, v) = 0 \iff u = v$
- $d(u, v) = d(v, u)$ - symmetry
- $d(u, z) \leq d(u, v) + d(v, z)$ - triangle inequality

The distance of the code is $d(C) = \min_{u \neq v} d(u, v) \quad (u, v \in C)$

Suppose the code alphabet A is an Abelian group with the zero element 0. Then the Hamming weight ($w(u)$) of a word u is the number of nonzero elements appearing in the word. Then the weight of the code is $w(C) = \min_{u \neq 0} w(u)$

Error-correcting code

When we receive a word that is not a code word, we correct it to the code word at the smallest Hamming distance from it.

The code K is t -error-correcting if it correctly corrects a code changed in at most t places. The code K is exactly t -error-correcting if it is t -error-correcting but not $(t + 1)$ -error-correcting.

Remark: for a code of minimal distance d we can certainly uniquely correct fewer than $d/2$ errors.

Hamming bound

A code C consisting of words of length n over an alphabet of q elements is t -error-correcting. Then for any two code words the sets of words at distance at most t from them are disjoint.

Since there are exactly $\binom{n}{j}(q-1)^j$ words at distance j from a code word, the Hamming bound for the number of code words at a given t is:

$$\#(C) \cdot \sum_{j=0}^t \binom{n}{j} (q-1)^j \leq q^n$$

If equality holds we speak of a perfect code.

1.4 Linear codes

Definition

A finite field K and A^n a linear space. Every subspace $K \leq A^n$ is called a linear code. If the subspace is k -dimensional, the distance of the code is d and $\#(A) = q$, then such a code is called an $[n, k, d]_q$ code.

For a linear code we assume that the messages to be coded are elements of K^k , that is, k -tuples formed from the elements of the code alphabet.

Generator matrix

Let us choose the $[n, k, d]_q$ linear coding over the finite field K as a (one-to-one) linear map:

$$G : K^k \rightarrow K^n$$

We can characterize this with a matrix, the so-called generator matrix.

Polynomial codes

For a linear code we can make the messages correspond to polynomials of degree lower than k over $\mathbb{F}_q$ (q -element finite field).

$$(a_0, a_1, \dots, a_{k-1}) \rightarrow a_0 + a_1x + \dots + a_{k-1}x^{k-1}$$

Let $g(x)$ be a fixed polynomial of degree m . Multiplying the polynomial $p(x)$ (message) by $g(x)$ we obtain a linear coding (since $p \rightarrow pg$ is one-to-one). Then the length of the code words is $n = k + m$. This type of linear coding is called polynomial coding.

Remark: We may assume that $g(x)$ is monic (its leading coefficient is a unit), and that the constant term is nonzero (if it were zero, the constant term would drop out in the product, so in the code the letter with index zero would never carry information)

CRC - Cyclic Redundancy Check

If in a polynomial code $g(x)|x^n - 1$, then we speak of a cyclic code. Then, if $a_0a_1 \cdots a_{n-1}$ is a code word, then so is $a_{n-1}a_0 \cdots a_{n-2}$, since:

$$a_{n-1} + a_0x + \cdots + a_{n-2}x^{n-1} = x \cdot (a_0 + a_1x + \cdots + a_{n-1}x^{n-1}) - a_{n-1}(x^n - 1)$$

is divisible by $g(x)$.

CRC encompasses the cyclic codes over $\mathbb{F}_2$. It is suitable only for error detection; the coding is the following: Take $p(x)x^m = (0, 0, \dots, 0, a_m, a_{m+1}, \dots, a_{n-1})$. Divide this by $g(x)$ with remainder. $p(x)x^m = q(x)g(x) + r(x)$. Then let the code word be: $p(x)x^m - r(x) = q(x)g(x)$, which is divisible by $g(x)$ and at the high degrees the letters of the original message are located. Checking the received word is simple: We check whether it is divisible by $g(x)$; if not, an error occurred.